

Ejercicios de parciales ordenados por su tipo, delante del ej. Está escrito de qué parcial salió.

Modelar, justificar y dejar en claro lo que sabemos. Evaluar tradeoffs, escribimos struct tanto.

[Ejercicios de parciales ordenados por su tipo, delante del ej. Está escrito de qué parcial salió.](#)

[Modelo de Concurrency](#)

[Verdadero o Falso](#)

[Es busy wait?](#)

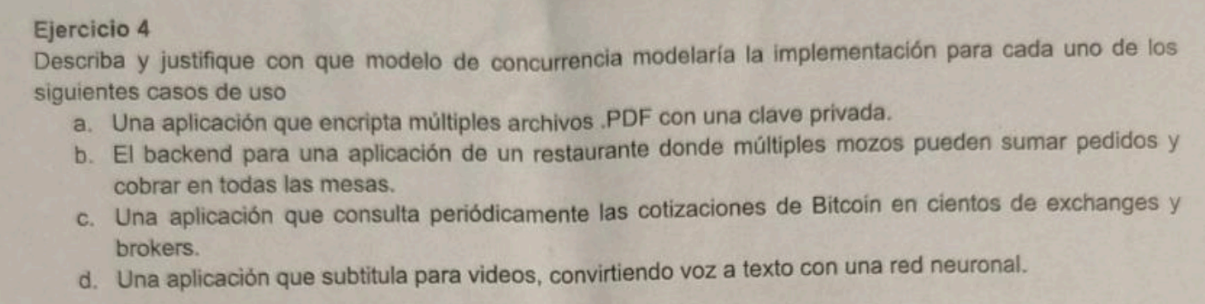
[Corrección de código](#)

[Implementación de algún problema con algún modelo específico](#)

[Redes de petri](#)

Modelo de Concurrency

[1c2025]



Ejercicio 4
Describe y justifique con que modelo de concurrency modelaría la implementación para cada uno de los siguientes casos de uso

- Una aplicación que encripta múltiples archivos .PDF con una clave privada.
- El backend para una aplicación de un restaurante donde múltiples mozos pueden sumar pedidos y cobrar en todas las mesas.
- Una aplicación que consulta periódicamente las cotizaciones de Bitcoin en cientos de exchanges y brokers.
- Una aplicación que subtitula para videos, convirtiendo voz a texto con una red neuronal.

- Fork join. La encriptación de cada archivo se puede realizar como una subtarea del objetivo principal del programa. Y en cuanto a la clave privada que es compartida, como es de solo lectura no requiere sincronización.
- Estado mutable compartido. las cuentas y los pedidos son datos compartidos entre los múltiples mozos, que dado el caso intentan modificar las mesas simultáneamente. Se debe garantizar que sea una acción atómica, por eso agregaremos exclusión mutua para serializar el acceso a esas variables.
- Programación asincrónica. La consulta de cotizaciones es una tarea intensiva en E/S (I/O-Bound) con alta latencia de red. El modelo Asincrónico es el más eficiente para esta alta concurrencia de E/S, ya que permite lanzar cientos de requests y ceder el control del hilo (await) mientras se espera la respuesta, evitando bloqueos costosos

- d. Fork join. La subtitulación y el procesamiento de una red neuronal son tareas de cómputo intensivo (CPU-Bound). El proceso se puede dividir en segmentos de video o frames que son independientes, lo que permite su procesamiento en paralelo para reducir significativamente el tiempo de ejecución.

[1c2025-recu2]

Ejercicio 3. Describa y justifique con que modelo de concurrencia modelaría la implementación para cada uno de los siguientes casos de uso

- Un sistema de consultas en tiempo real sobre un conjunto de miles de tablas.
- El backend para un juego estilo MMOG (massive multiplayer online game)
- Un crawler que descarga contenidos de internet para entrenar una red neuronal.
- Una aplicación que dado un conjunto de videos de ejemplo, genera un nuevo video donde los mismos personajes dicen lo que se les indica con un prompt ("deepfake").

- Fork-join. Tarea de cómputo intensivo, dividimos las consultas a las tablas en subtareas y ejecutamos en paralelo para reducir la latencia al final. Además aplicar este método que es determinístico termina
- Actores. El modelo de actores proporciona el aislamiento completo del estado y permite que las entidades se comuniquen de forma asíncrona mediante mensajes, lo cual es altamente escalable y evita los locks y las condiciones de carrera
- Programación asíncrona. La descarga es una actividad de i-o. El modelo Asíncrono es el más eficiente porque le permite al sistema lanzar miles de peticiones y, cuando una queda esperando la respuesta de la red, cede el control del thread (`await`) para que otra tarea pueda avanzar .
- fork-join. El procesamiento de videos es una tarea de cómputo intensivo, se puede dividir el procesamiento de dichos videos en subtareas, por ejemplo procesar por frames y luego unir todas al finalizar dicho procesamiento.

[1c2025-recu1]

Ejercicio 3

Describa y justifique con que modelo de concurrencia modelaría la implementación para cada uno de los siguientes casos de uso

- Un sistema de subastas en línea donde múltiples usuarios ofrecen por ítems en tiempo real, se debe actualizar el precio del producto y notificar a los usuarios involucrados.
- Una aplicación de una pinturería que aplica filtros en una imagen para ver cómo quedarían distintos colores sobre una pared.
- Una memoria cache utilizada por un modelo de Machine Learning usado para reconocimiento facial.
- Una herramienta que dado N imágenes (donde N es muy grande) las junta en grupos de 100 y las transforma a PDF.

- Actores o Estado mutable compartido Actores: Este modelo es ideal porque el ítem de subasta puede ser modelado como un Actor que encapsula el precio (su estado) y maneja los eventos de oferta y notificación de forma asíncrona. Esto es altamente escalable y evita locks y condiciones de carrera al serializar los mensajes en el mailbox.

Estado Mutable Compartido: Si se opta por este modelo, el precio del producto es el dato crítico compartido que debe actualizarse con Exclusión Mutua (**Mutex**) para garantizar que la actualización de la oferta sea una transacción atómica

- b. fork join. Se puede dividir el problema en subtareas, como un pixel por tarea, y dado a que es determinístico siempre se va a obtener el mismo resultado.
- c. Estado mutable compartido. la caché es un recurso compartido, que se debe actualizar entonces para acceder se debería bloquear el acceso cuando este está siendo editado. podemos usar un **rwLock** para que múltiples hilos lean concurrentemente y solo uno impone la lectura.
- d. Fork join. La transformación de cada grupo de 100 imágenes a PDF es una tarea de cómputo y E/S de archivos que es independiente de los demás grupos. El modelo Fork-Join permite la ejecución paralela de estos lotes de trabajo para reducir el tiempo total de procesamiento.

[2c2023]

Ejercicio 1

Enumere y justifique qué modelo de concurrencia es más conveniente utilizar para cada uno de los siguientes casos de uso

- Calcular productos de matrices para entrenar redes neuronales
 - Realizar llamadas a diferentes APIs y combinar sus resultados
 - Procesar los archivos de log de acceso de un sitio web muy concurrido
 - Backend de una app para editar documentos de forma colaborativa en línea, como un Google Doc
-
- a. Fork-join. es una tarea de cómputo intensivo, se puede subdividir en subtareas y realizar el procesamiento paralelamente
 - b. Programación asincrónica. Las llamadas a las apis son tareas de alta latencia de entrada y salida, así que podemos lanzar llamadas concurrentes con **await** y poder ir entregando el control del thread mientras se espera la reapuesta.
 - c. Programación asincrónica. La lectura de grandes archivos de log (E/S de archivos o de red) es una operación que puede ser lenta y bloqueante. El modelo Asincrónico es útil para la parte de lectura de E/S, permitiendo que el sistema siga siendo responsivo mientras espera que los datos del log estén disponibles.
 - d. Actores. Un backend de videojuego requiere gestionar muchas entidades concurrentes (jugadores) con su propio estado. El modelo de Actores es adecuado porque proporciona aislamiento de estado y escalabilidad, comunicando las entidades solo por mensajes. Si el juego es multijugador, se podría usar Estado Mutable Compartido si los jugadores interactúan con recursos críticos comunes, requiriendo **Mutex** para asegurar la integridad de dichos recursos.

[1c2024]

1. Revisando el diseño aplicado en algunos proyectos, se encontró el uso de las siguientes herramientas para resolver problemas de concurrencia. Para cada uno de los problemas enumere ventajas o desventajas de utilizar la solución propuesta y menciona cual utilizaría usted.

- Renderizado de videos 3D en alta resolución, utilizando programación asincrónica.
- Aplicación que arma una nube de palabras a partir de la API de Twitter, utilizando barriers y mutex.
- Una aplicación para realizar una votación en vivo para un concurso de televisión, optimizada con Vectorización.

a. Programación asincrónica. Ventajas: Evita bloqueos por I/O, muy eficiente para aplicaciones reactivas

Desventajas: Puede volverse difícil de leer si se abusa de callbacks o promesas mal estructuradas

Yo utilizaría fork-join dado que el cómputo intensivo se puede paralelizar para maximizar la velocidad.

b. Barriers y mutex. Ventajas:

Desventajas: Las solicitudes a la API son operaciones de alta latencia de red (I/O-Bound). El uso de barriers es ineficiente porque obligaría a todos los threads a esperar por la respuesta más lenta de la red antes de poder continuar con la siguiente fase (como el procesamiento). Esto desperdicia recursos

Yo utilizaría Programacion asincronica

c. Vectorizacion. Ventajas:

Desventajas: La Vectorización solo acelera cálculos numéricos; no es una herramienta de sincronización. La votación requiere modificar un estado compartido (el contador de votos). Usar solo vectorización resultaría en condiciones de carrera y conteos incoherentes.

Yo utilizaría Estado mutable compartido

5. Describa y justifique con que modelo de concurrencia modelaría la implementación para cada uno de los siguientes casos de uso

- a. Convertir un conjunto extenso de archivos de .DOC a .PDF
- b. El backend para una aplicación de preguntas & respuestas competitiva al estilo Menti o Kahoot.
- c. Una memoria caché utilizada para reducir la cantidad de requests en un servidor web a una base de datos.
- d. Una API HTTP que ejecuta un modelo de procesamiento de lenguaje natural para clasificar el sentimiento de un mensaje.

- a. Fork-join. Se puede paralelizar en subtareas el procesamiento de archivos.
- b. Actores o canales-mensajes. El modelo de Actores es ideal porque proporciona aislamiento de estado (cada sala o usuario es un actor) y gestiona la comunicación por mensajes asincrónicos, lo cual es altamente escalable .
- c. Estado mutable compartido. Con un RwLock los distritos hilos leen concurrentemente y solo pone exclusión mutua para la acción de escritura
- d. Programación asincrónica. Api, es una tarea de alta concurrencia de entrada y salida. El modelo Asincrónico es el más eficiente para esta tarea, ya que minimiza el overhead de los hilos y permite al sistema ceder el control del thread mientras espera datos de la red

Verdadero o Falso

[1c2025]

Ejercicio 5

Verdadero o Falso. Justifique

- a. Procesos, hilos y tareas asincrónicas todos poseen espacios de memoria independientes entre sí.
- b. El scheduler del sistema operativo puede detener una tarea asincrónica puntual y habilitar la ejecución de otra para el mismo proceso.
- c. En vectorización, tanto las operaciones verticales como las operaciones horizontales devuelven resultados vectoriales
- d. Un hilo esperando sobre una condvar sólo puede despertarse cuando otro hilo hace signal de la misma.

- a. Falso, los hilos y las tareas comparten el espacio de memoria del proceso en el que corren.
- b. Falso, El scheduler del SO solo gestiona la preempción de hilos (threads). Las tareas asincrónicas (Futures) operan bajo un modelo cooperativo. Esto significa que el scheduler no puede interrumpir una tarea asincrónica; la tarea debe ceder el control voluntariamente al Executor (usando await).
- c. Falso, las operaciones verticales si lo hacen pero las horizontales no, por lo general devuelven un valor escalar.

- d. Falso . Un hilo puede despertar también por un fenómeno conocido como "spurious wake-up" (despertada espuria o falsa). Es por esto que la condición de espera debe ser verificada con un bucle while al despertar, para asegurar que la condición realmente se cumpla.

[1c2025-recu2]

Ejercicio 2. Verdadero o Falso. Justifique

- a. Las tareas asincrónicas son especialmente útiles para procesos de cómputo intensivo.
- b. El estado interno de un actor puede accederse desde otros actores mediante locks
- c. Programando en el modelo fork-join se debe tener especial cuidado en evitar condiciones de carrera.
- d. En el estado mutable compartido los hilos deben verificar periódicamente por actualizaciones de dicho estado

- a. Falso. Las tareas asincrónicas son útiles para procesos de E/S (I/O-Bound), donde hay mucha espera o latencia. Para el cómputo intensivo (CPU-Bound), el modelo ideal es Fork-Join (Paralelismo).
- b. Falso. El estado de un actor está aislado y solo puede ser mutado por el actor mismo al procesar mensajes. Los actores solo se comunican enviando mensajes, prohibiendo el acceso directo con locks.
- c. Falso. El modelo Fork-Join se aplica a tareas independientes que no comparten estado mutable. Si se usa correctamente, el modelo es determinístico, no requiere sincronización y, por lo tanto, no presenta condiciones de carrera
- d. Falso. La verificación periódica es un Busy Wait (espera activa), que es ineficiente y consume CPU. El modelo debe utilizar mecanismos bloqueantes (Condvar::wait() o Semáforos) para que los hilos esperen pasivamente una notificación de cambio de estado .

[1c2025-recu1]

Ejercicio 2

Verdadero o Falso. Justifique

- a. En un ambiente de ejecución con una única CPU, un conjunto de hilos de procesamiento intensivo tomarán un tiempo de ejecución significativamente menor a un conjunto de tareas asincrónicas que ejecuten el mismo procesamiento.
- b. Programando en el modelo fork-join se debe tener especial cuidado en evitar condiciones de carrera.
- c. En el estado mutable compartido los hilos deben verificar periódicamente por actualizaciones de dicho estado
- d. El estado interno de un actor se encuentra protegido por una exclusión mutua.

- a. Falso. en una unica cpu solo hay concurrencia, en forma de intercalacion pero no paralelismo real.
- b. Falso. El modelo Fork-Join se utiliza para tareas independientes que no comparten estado mutable. Por diseño, es determinístico y no requiere sincronización, por lo que no presenta condiciones de carrera.
- c. Falso. La verificación periódica constituye un Busy Wait (espera activa), que es ineficiente y consume CPU. El modelo debe utilizar mecanismos bloqueantes

(Condvar::wait() o Semáforos) para que los hilos esperen pasivamente una notificación de cambio de estado.

- d. Falso, el estado interno de un actor esta aislado por una exclusion mutua per se, sino una enxculision mutua interna proporcionada por su mailbox, es algo interno.

[1c2024]

2. Programación asincrónica. Elija verdadero o falso y explique brevemente por qué:

- El encargado de hacer poll es el thread principal del programa.
 - El método poll es llamado únicamente cuando la función puede progresar.
 - El modelo piñata es colaborativo.
 - La operación asincrónica inicia cuando se llama a un método declarado con async.
- a. Falso. El encargado de llamar a poll es el Executor del runtime asincrónico (como Tokio), que es una abstracción que puede correr en cualquier hilo gestionado por el runtime, no necesariamente el hilo principal .
- b. Falso. El poll es llamado por el Executor cuando considera que un recurso podría estar disponible. La llamada no garantiza progreso; el Future puede devolver Pending si la operación de E/S aún no está lista, sin haber avanzado en ese momento.
- c. Falso. El modelo async/await (o modelo piñata) es colaborativo (cooperativo) porque las tareas ceden el control voluntariamente al Executor utilizando la expresión await. El Executor solo cambia de tarea cuando la tarea actual se suspende intencionalmente
- d. Falso. Al llamar a una función async, esta no se ejecuta de forma inmediata. En su lugar, retorna inmediatamente un Future, que es la estructura que encapsula la operación pendiente. La ejecución real comienza recién cuando el Executor llama a poll() sobre ese Future.

[2c2024]

4. Verdadero o Falso. Justifique

- a. Procesos, hilos y tareas asincrónicas poseen espacios de memoria independientes.
 - b. El scheduler del sistema operativo puede detener una tarea asincrónica puntual y habilitar la ejecución de otra para el mismo proceso.
 - c. Tanto los threads, como las tareas asincrónicas disponen de un stack propio.
 - d. En un ambiente de ejecución con una única CPU, un conjunto de hilos de procesamiento intensivo tomarán un tiempo de ejecución significativamente menor a un conjunto de tareas asincrónicas que ejecuten el mismo procesamiento.
-
- a. Falso, Los Hilos (threads) y las Tareas Asincrónicas comparten el espacio de memoria (el heap y los segmentos de memoria global) del proceso al que pertenecen. Solo los Procesos tienen espacios de memoria totalmente independientes
 - b. Falso. El scheduler del SO solo gestiona la preempción de Hilos. Las tareas asincrónicas utilizan un modelo cooperativo gestionado por un Executor, lo que significa que el scheduler del SO no puede interrumpirlas; la tarea debe ceder el control mediante await
 - c. Falso. Los Hilos tienen su propio stack. Las Tareas Asincrónicas (Futures) son más livianas y no tienen un stack tradicional; almacenan su estado local (máquina de estados) para ser reanudadas, lo que es distinto a un stack de hilo completo.
 - d. Falso. En una única CPU, solo hay concurrencia (intercalación), no paralelismo real. Los hilos intensivos en CPU tienen un alto costo por context switching del SO. Las tareas asincrónicas son más livianas y rápidas de manejar, por lo que el tiempo de ejecución no sería significativamente menor para los hilos

[2c2024-recu1]

Ejercicio 4

Verdadero o Falso. Justifique

- a. El estado interno de un actor se encuentra protegido por una exclusión mutua.
 - b. La llamada al método poll de un Future es bloqueante.
 - c. Programando en el modelo fork-join se debe tener especial cuidado en evitar condiciones de carrera.
 - d. Un hilo esperando sobre una condvar sólo puede despertarse cuando otro hilo hace signal de la misma.
-
- a. Falso. No es una exclusion mutua como la solemos ver, pero si es una exclusion mutua interna, cada actor la maneja por separado con su propio mailbox.

- b. Falso. El método poll nunca bloquea . Su función es verificar si el Future está listo (Ready) o si está pendiente (Pending), permitiendo que el Executor trabaje en otra cosa .
- c. Falso. El modelo Fork-Join se aplica a tareas independientes que no comparten estado mutable. Por diseño, es determinístico, no requiere sincronización y, por lo tanto, no presenta condiciones de carrera
- d. Falso. Un hilo también puede despertar debido a un "spurious wake-up" (despertada espuria o falsa) . Por esta razón, la condición debe ser rechequeada con un bucle while al despertar.

Es busy wait?

-> dependiendo el contexto, si estás en un dispositivo, te quedas sin batería o quemas el hardware.

[1c2025]

```
1. for _ in 0..MINERS {  
    let copper = Arc::clone(&resource);  
    thread::spawn(move || loop {  
        let mined_amount = rand::thread_rng().gen_range(1..10);  
        *copper.write().expect("failed to mine") += mined_amount;  
        let delay = rand::thread_rng().gen_range(3000..7000);  
        thread::sleep(Duration::from_millis(delay));  
    });  
}
```

No es busy wait, podemos afirmar que el método expect es bloqueante, por lo tanto, el thread espera de manera pasiva. Si un thread ya posee el recurso copper, el thread minero se suspende y espera pasivamente a ser notificado.

[1c2025]

```

2. fn philosopher(id: usize, first_chopstick: Arc<Semaphore>, second_chopstick
   Arc<Semaphore>) {
    loop {
        println!("Philosopher {}: thinking", id);
        thread::sleep(Duration::from_millis(rng.gen_range(500..1500)));

        loop {
            println!("Philosopher {}: taking first chopstick", id);
            let first_access = first_chopstick.acquire();

            println!("Philosopher {}: attempting second chopstick", id);
            match second_chopstick.try_acquire() {
                Ok(second_access) => {
                    println!("Philosopher {} eating", id);
                    let delay = rand::thread_rng().gen_range(3000..7000);
                    thread::sleep(Duration::from_millis(delay));
                    break;
                }
                Err(_) => {
                    println!("Philosopher {} was unsuccessful", id);
                    drop(first_access);
                }
            }
        }
    }
}

```

si es busy wait dado que cae en un método de verificación y
reintento rápido

Durante el intento del segundo palillo realiza el try-acquire que no
es bloqueante y en el caso de no estar disponible, reintenta de
inmediato toda la secuencia completa.

[1c2025]


```

3. struct Data {
    value: Option<i32>,
}

fn main() {
    let data = Arc::new(Mutex::new(Data { value: None }));

    let c1 = Arc::clone(&data);
    let t1 = thread::spawn(move || {
        // does some work
        let mut lock = c1.lock().unwrap();
        lock.value = Some(42);
        ...
    });

    let c2 = Arc::clone(&data);

```

```

        let t2 = thread::spawn(move || {
            loop {
                let lock = c2.lock().unwrap();
                if lock.value.is_some() {
                    println!("Valor obtenido: {}", lock.value.unwrap());
                    break;
                }
            }
        });

    t1.join().unwrap();
    t2.join().unwrap();
}

```

Es Busy wait, por que el hilo t2 esta implementado con un bucle de verificación constante y rápida de una condición, en lugar de utilizar un mecanismo de bloqueo pasivo.

[1c2024]

```

for _ in 0..MINERS {
    let lithium = Arc::clone(&mineral);
    thread::spawn(move || loop {
        let mined = rand::thread_rng().gen();
        let random_result: f64 = rand::thread_rng().gen();

        *lithium.write().expect("failed to mine") += mined;
        thread::sleep(Duration::from_millis((5000 as f64 *
random_result) as u64));
    })
}

```

```

for _ in 0..MINERS {
    let lithium = Arc::clone(&mineral);
    let batteries_produced = Arc::clone(&resources);
    thread::spawn(move || loop {
        let mut lithium = lithium.write().expect("failed");
        if lithium >= 100 {
            lithium -= 100;
            batteries_produced.write().expect("failed to produce") += 1
        }
        thread::sleep(Duration::from_millis(500));
    })
}

```

1. No es Busy wait, el hilo esta usando mecanismos de espera pasiva con la funcion bloqueante expect y cesion de cpu con sleep.
2. Si es Busy wait, porque se queda esperando q que lithium se libere para realizar la operacion.

[2c2024]

1. Para cada uno de los siguientes fragmentos de código indique si es o no es un busy wait. Justifique en cada caso.

```
1. loop {  
  match TcpStream::connect("127.0.0.1:8080") {  
    Ok(mut stream) => {  
      stream.write_all(message.as_bytes()).expect("error")  
    }  
    Err(_) => {  
      let random_result: f64 = rand::thread_rng().gen();  
      thread::sleep(  
        Duration::from_millis((5000 as f64 * random_result) as u64)  
      );  
    }  
  }  
}
```

```
2. loop {  
  let random_result: f64 = rand::thread_rng().gen();  
  thread::sleep(Duration::from_millis(random_result as u64));  
  let mut items = self.pending_acks.lock().unwrap();  
  let now = Instant::now();  
  let mut i = 0;  
  while i < items.len() {  
    if items[i].expiration <= now {  
      if items[i].item_type == "ACK" {  
        let _ = items.remove(i);  
        drop(items);  
        self.send_result_interfaces();  
        break;  
      }  
    } else {  
      i += 1;  
    }  
  }  
}
```

```
3. for _ in 0..MINERS {  
  let copper = Arc::clone(&resource);  
  thread::spawn(move || loop {  
    let mined_amount = rand::thread_rng().gen_range(1..10);  
    *copper.write().expect("failed to mine") += mined_amount;  
    let delay = rand::thread_rng().gen_range(3000..7000);  
    thread::sleep(Duration::from_millis(delay));  
  });  
}
```

1. Si es busy wait, el hilo está atrapado en un loop infinito, que intenta activamente establecer una conexión. y luego **repite el bucle inmediatamente** para reintentar la conexión. Este patrón de **reintento activo y continuo** consume recursos en la espera.
2. Si es busy wait, El hilo entra en un loop infinito que, aunque tiene un `thread::sleep()` inicial, en cada iteración adquiere un lock (`pending_acks.lock()`) y realiza un chequeo activo

(while i < items.len(), if items[i].expiration < now) sobre la condición de expiración. El propósito principal del hilo es verificar periódicamente el estado de un recurso, lo cual es ineficiente y es la definición de espera activa, ya que no hay un mecanismo de notificación pasiva que despierte al hilo solo cuando hay un cambio.

3. no es busy wait, El hilo utiliza mecanismos de espera pasiva y cesión de CPU.

Corrección de código

-> ojo con mezclar actores con convars
[1c2025]

Ejercicio 2

Dado el siguiente fragmento de código, indique el nombre del problema que modela. Indique si la implementación es correcta o describa cómo mejorarla.

```
fn main() {
    const N: usize = 5;

    let producers_waiting = Arc::new(Semaphore::new(0));
    let consumer_done = Arc::new(Semaphore::new(0));
    let data = Arc::new(Mutex::new(None))

    let producers_waiting_clone = producers_waiting.clone();
    let consumer_done_clone = consumer_done.clone();

    let consumer = thread::spawn(move || loop {
        println!("[Consumer] Sleeping...");
        producers_waiting_clone.acquire();

        println!("Doing my job with data {}...", data.lock().expect("can read"));
        thread::sleep(Duration::from_secs(2));

        consumer_done_clone.release();
        println!("Job finished");
    });

    let producers: Vec<_> = (0..N).map(|id| {
        let producers_waiting_clone = producers_waiting.clone();
        let consumer_done_clone = consumer_done.clone();

        thread::spawn(move || loop {
            let delay = rand::thread_rng().gen_range(3000..7000);
            thread::sleep(Duration::from_millis(delay));
            println!("[Producer {}] Arrived", id);
            producers_waiting_clone.release();

            *data.lock().expect("can't set data") = Some(id)

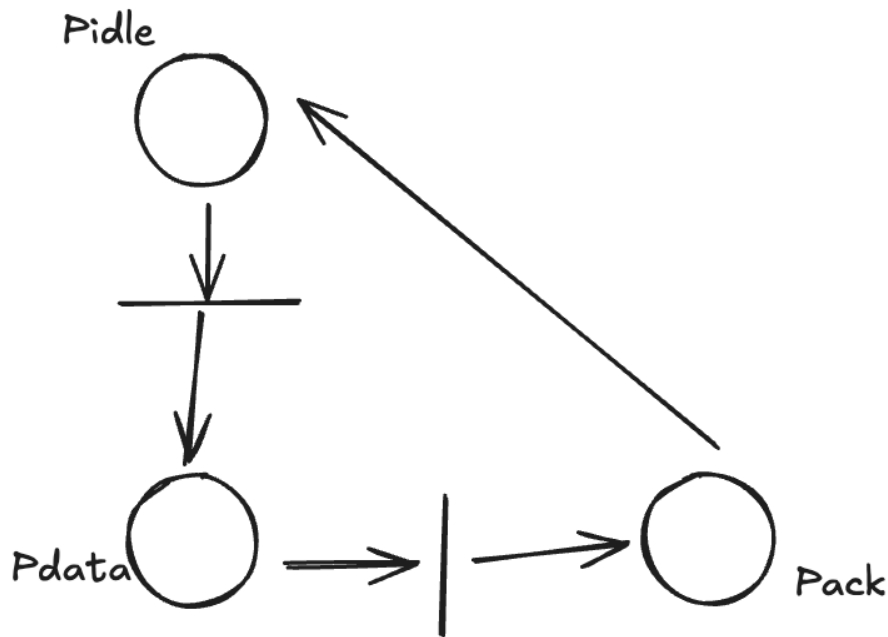
            println!("[Producer {}] waiting to be consumed", id);
            consumer_done_clone.acquire();

            println!("[Producer {}] Leaving", id);
        })
    }).collect();

    consumer.join().unwrap();
    for p in producer { p.join().unwrap(); }
```

Modele una red de Petri para el problema del punto anterior. Si hubiera propuesto mejoras, inclúyalas.

Se podría implementar el modelo de canales y mensajes donde tenemos un canal unidireccional, el canal transfiere el ownership del dato y se bloquea al momento de recibir y de mandar solo si el canal está lleno.



[1c2025-recu2]

Ejercicio 1. Una IA generó los siguientes fragmentos de código. Identifique problemas y proponga soluciones.

a. <pre> struct ResourceGuard { guard: Semaphore } impl Actor for ResourceGuard { type Context = Context<Self>; } impl Handler<Acquire> for ResourceGuard { type Result = (); fn handle(&mut self, _msg: Acquire, _ctx: &mut Context<Self>) -> Self::Result { self.guard.acquire(); } } impl Handler<Release> for ResourceGuard { type Result = (); fn handle(&mut self, _msg: Release, _ctx: &mut Context<Self>) -> Self::Result { self.guard.release(); } } </pre>	b. <pre> impl Handler<Process> for Resource { type Result = (); fn handle(&mut self, msg: Process, _ctx: &mut Context<Self>) -> Self::Result { if !self.resourceReady { _ctx.address().do_send(msg) } // ... } } </pre>
--	--

- a. Eliminar el Semáforo Bloqueante: Si se busca control de acceso, se debe usar un mecanismo que no bloquee el Arbiter. Usar Mensajes Asíncronos: Si se necesita un Actor que controle el acceso, este debe mantener su estado internamente (`recurso_libre: bool`) y usar la cola de espera de su mailbox para serializar las peticiones, o usar `Future::await` en una primitiva asíncronica (si estuviera disponible).
- b. Poner el mensaje en una cola interna, se intenta acceder a un recurso que no está listo. Sustituir el Reenvío por Espera Pasiva: El Actor debe suspenderse pasivamente y esperar a que

el estado cambie, en lugar de bombardearse a sí mismo con mensajes.

Usar un Future con Delay: Si el Actor necesita esperar activamente a que algo esté listo, debe usar un mecanismo de espera asincrónico (ej., un Future o un Timer) o suspender su lógica de procesamiento principal hasta ser notificado por otro Actor.

Lógica Inversa Correcta: Si el Actor no está listo, el mensaje debe ser puesto en una cola de espera interna y no re-enviado inmediatamente.

[1c2025-recu1]

Ejercicio 1. A partir de los siguientes fragmentos de código, identifique problemas y proponga soluciones.

```
a.
struct Producer {
    buffer: Arc<(Condvar, Mutex<Vec<i32>>>>,
}
impl Actor for Producer {
    type Context = Context<Self>;
}
impl Handler<Produce> for Producer {
    type Result = ();
    fn handle(&mut self, msg: Produce, _ctx: &mut Context<Self>) -> Self::Result {
        let (cvar, lock) = &*self.buffer;
        let mut guard = cvar.wait_while(lock.lock().unwrap(), |buf| { buf.len() >= 10 })
        *guard.push(msg.value)
    }
}
impl Handler<Consume> for Producer {
    type Result = ();
    fn handle(&mut self, _msg: Consume, _ctx: &mut Context<Self>) -> Self::Result {
        let (cvar, lock) = &*self.buffer;
        let mut guard = cvar.wait_while(lock.lock().unwrap(), |buf| { buf.len() == 0 })
        return *guard.pop();
    }
}
```

a. Se está mezclando el modelo de actores con convars. lo mas logico seria que si necesitas esperar una condición dentro del Modelo de Actores, debes usar mecanismos **asincrónicos** como *Futures* o lógica de *re-envío* (*do_send* a sí mismo o a otro actor),

[1c2025-recu1]

```

b.
impl Handler<PrepararCafe> for Barista {
  fn handle(&mut self, _msg: PrepararCafe, _ctx: &mut Context<Self>) {
    loop {
      for maquina in self.maquinas.iter() {
        match maquina.send(EstasLibre).await {
          Ok(true) => {
            println!("Barista. Máquina libre, preparando café");
            // Preparar café (sin esperar resultado)
            maquina.do_send(PrepararCafe);
            return;
          },
          Ok(false) => {
            println!("Barista. Sigo buscando máquina libre");
          },
          Err(e) => {
            println!("Barista. Error: {}", e);
          }
        }
      }
    }
  }
}

```

b. se hace un await dentro del handler, suspende al actor pero bloquea al arbiter, podemos aplicar un do_send o hacer un rediseño del programa

Implementación: Crear un Actor Gerente de Máquinas que mantenga el estado de las máquinas. Cuando una máquina se libera, el Gerente debe notificar (do_send) a un Barista en espera con un mensaje MaquinaDisponible. El Barista solo debe intentar buscar o trabajar cuando reciba esta notificación

[1c2024]

```

pub struct SynchronizationStruct {
    mutex: Mutex<i32>,
    cond_var: Condvar,
}

impl SynchronizationStruct {
    pub fn new(size: u16) -> SynchronizationStruct {
        SynchronizationStruct {
            mutex: Mutex::new(size),
            cond_var: Condvar::new(),
        }
    }

    pub fn function_1(&self) {
        let mut amount = self.mutex.lock().unwrap();
        if *amount <= 0 {
            amount = self.cond_var.wait(amount).unwrap();
        }
        *amount -= 1;
    }

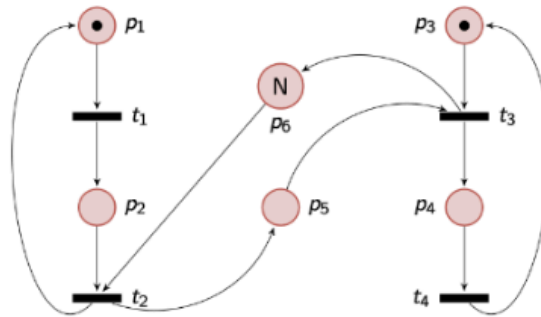
    pub fn function_2(&self) {
        let mut amount = self.mutex.lock().unwrap();
        *amount += 1;
        self.cond_var.notify_all();
    }
}

```

a.

[1c2024]

5. Dados la siguiente red de Petri y fragmento de código, indique el nombre del problema que modelan. Indique si la implementación es correcta o describa cómo mejorarla.



```
fn main() {  
    let sem = Arc::new(Semaphore::new(0));  
    let buffer = Arc::new(Mutex::new(Vec::with_capacity(N)));  
  
    let sem_cloned = Arc::clone(&sem);  
    let buf_cloned = Arc::clone(&buffer);  
    let t1 = thread::spawn(move || {  
        loop {  
            // heavy computation  
            let random_result: f64 = rand::thread_rng().gen();  
            thread::sleep(Duration::from_millis((500 as f64 *  
random_result) as u64));  
  
            buf_cloned.lock().expect("").push(random_result);  
            sem_cloned.release()  
        }  
    });  
  
    let sem_cloned = Arc::clone(&sem);  
    let buf_cloned = Arc::clone(&buffer);  
    let t2 = thread::spawn(move || {  
        loop {  
            sem_cloned.acquire();  
            println!("{}", buf_cloned.lock().expect("").pop());  
        }  
    });  
  
    t1.join().unwrap();  
    t2.join().unwrap();  
}
```

kjn1

[2c2024-recu1]

Ejercicio 2

En una revisión de código se encontró el siguiente fragmento

```
const N:usize = 10;
fn main() {
    let buffer = Arc::new(Mutex::new(Vec::<u32>::with_capacity(N)));
    let buffer_local = buffer.clone();
    let handle = thread::spawn(move || {
        loop {
            let mut buf = buffer_local.lock().unwrap();
            if buf.len() < N {
                buf.push(rand::thread_rng().gen());
            } else {
                drop(buf);
                thread::sleep(Duration::from_secs(1));
            }
        }
    });

    loop {
        let mut buf = buffer.lock().unwrap();
        if !buf.is_empty() {
            println!("{}", buf.pop().unwrap());
        } else {
            drop(buf);
            thread::sleep(Duration::from_secs(1));
        }
    }

    handle.join().unwrap();
}
```

- Describe qué problemas presenta este código y proponga una solución
- Describe por qué son necesarias las llamadas a drop en esta implementación.

Implementación de algún problema con algún modelo específico

[1c2025]

Ejercicio 3

Se quiere abrir una estación de servicio YPF con 10 surtidores. Por el momento solamente se logró la habilitación de 3 tanques internos de nafta, el objetivo es a futuro poder habilitar más. Los surtidores atienden a los autos que vienen a cargar combustible, pero deben acceder a los tanques internos de un surtidor a la vez. Además, cuando llega el camión cisterna de YPF a recargar el combustible disponible en la estación de servicio, este debe asegurarse de que ningún tanque esté siendo usado por un surtidor ya que eso representa un riesgo de seguridad.

Diseña el sistema utilizando el modelo de actores, y para cada entidad define cuáles son los estados internos y los mensajes que intercambian.

3 tanques de nafta, con 10 surtidores.
surtidores -> autos

autos-> cargan combustible

Camion cisterna -> recarga pero con la condicion de que ningun tanque esté siendo usado por un surtidor.

Modelo de actores:

Tanque (x3):nivel: f64, ocupado_por_surtidor:

Option<Addr<Surtidor>>, esta_en_recarga: bool

SolicitarUso(Litros, SurtidorAddr): Procesamiento Secuencial

(Exclusión). Si ocupado_por_surtidor es None y esta_en_recarga es false, marca la ocupación y realiza el descuento. Responde al

Surtidor con UsoAceptado. **Recargar(Cantidad, CamionAddr):**

Procesamiento Secuencial (Seguridad). Si ocupado_por_surtidor es

None (nadie lo usa), marca esta_en_recarga = true, realiza la recarga y notifica al Camión. Si está ocupado, notifica

TanqueOcupado

Surtidores:tanque_asignado: Addr<Tanque>

CargarAuto(Litros): Envía un mensaje SolicitarUso al Tanque asignado y espera el resultado para despachar el combustible o denegar la carga

Camión cisterna: tanques_pendientes: Vec<Addr<Tanque>>

IniciarRecarga(tanque_addr): Envía un mensaje Recargar al Tanque y espera el resultado. Si el tanque responde TanqueOcupado, vuelve a intentar más tarde

Auto: no requiere de un estado interno, pero dado el caso se puede poner como tanque vacío y que mande un mensaje de cargar nafta y tanque lleno una vez que termine.

[1c2025-recu2]

Ejercicio 4. Dado el siguiente enunciado

En un salón de videojuegos, los mismos se pagan con una tarjeta recargable. En el mismo salón se festejan cumpleaños. En el caso de estos, se le da a cada invitado una tarjeta que descuenta dinero de un pool compartido entre todos.

- Modelarlo con actores, describiendo sus estados y mensajes en pseudocódigo de Rust, y cómo reaccionan.
- Modelarlo con una red de Petri.

Pool dinero:saldo: u32

Gastar(monto, TarjetaAddr): Procesamiento Secuencial. Si saldo $\geq$ monto, descuenta el monto y notifica a la Tarjeta el éxito. Si el saldo es insuficiente, notifica el fallo.

Recargar(monto): Incrementa el saldo.

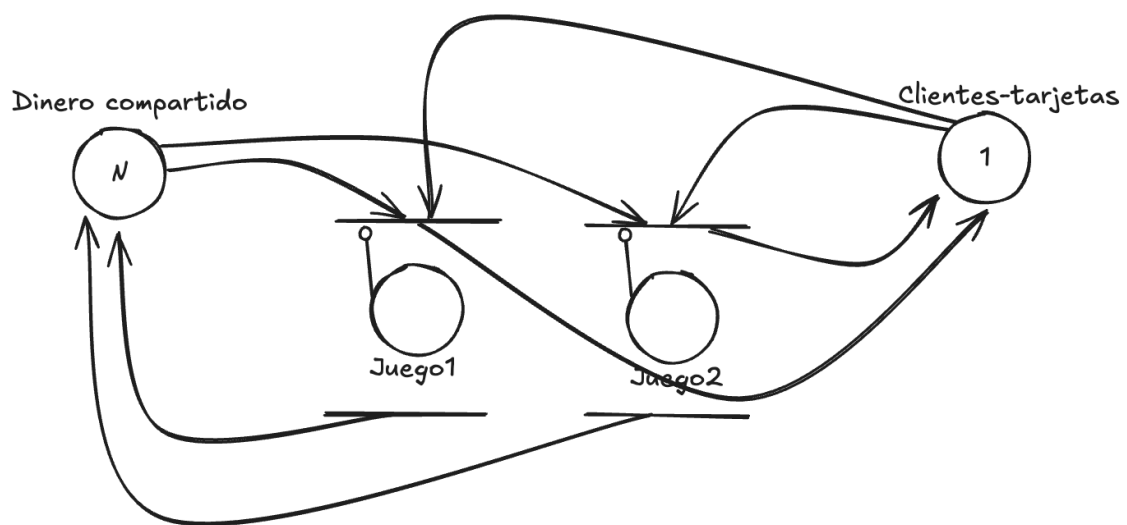
Tarjeta: pool_addr: Addr<PoolDinero>, costo_juego: u32

Jugar(): Envía el mensaje **Gastar** al PoolDinero con el costo del juego.

RespuestaGasto(exito): Procesa el resultado para permitir o denegar el juego al invitado

VideoJuego: costo: u32

IniciarJuego(TarjetaAddr): Recibe la solicitud y delega el gasto en la Tarjeta (la interfaz del invitado).



[1c2025-recu1]

Ejercicio 4

En una oficina sobre la 9 de Julio trabaja una empresa que posee una serie de máquinas de café automáticas distribuidas en todas las áreas, a lo largo de los 10 pisos que alquila.

Para comprar café en las máquinas, cada empleado tiene una tarjeta cargada con créditos. Sin embargo, los créditos son compartidos por equipo para mejor control de cada gerencia.

Los créditos se cargan el primer día hábil de cada mes, y además cada gerente tiene la posibilidad de agregar montos extra a lo largo del mes como bonus cuando considera que sus empleados deben ser recompensados.

Modele el sistema con actores, describiendo sus estados y mensajes en pseudocódigo de Rust. Describa cómo reaccionan ante los mensajes que reciben (en prosa).

Especifique cómo asegura su modelo el caso del "doble gasto" si dos tarjetas asociadas al mismo equipo se utilizan en dos máquinas al mismo tiempo.

Maquinas de cafe

-> Recurso compartido: los créditos

El mailbox del actor creditos actua como el lock.

Créditos: saldo: u32

Consumir(monto, maquinaAddr), Recargar(monto) esta ultima puede servir para cargar el bonus también.

Maquina de cafe: creditos_addr: Addr<Creditos>, equipo_id: u32

SolicitarCafe(monto, EmpleadoID), ResultadoConsumo(exito)

Gerente: creditos_addr: Addr<Creditos>

CargarMes(monto), AplicarBonus(monto)

Tarjeta empleado: maquina_addr: Addr<Maquina>

ComprarCafe()

[2c2023]

2. Desarrollar un pseudocodigo en Rust que solucione

- Se tiene 100 links de distintas páginas.
- Simular el tiempo de espera usando sleep y rand.
- No se pueden tener más de N threads activos a la vez (para tener latencia baja).
- Se desea calcular el tiempo promedio total.

// Pseudocódigo Rust

```

use std::sync::{Arc, Mutex};
use std::thread;
use std::time::{Duration, Instant};
use std_semaphore::Semaphore; // Asumiendo un crate de semáforos
use rand::{Rng, thread_rng};

// Constantes
const N_SITES: usize = 100;
const MAX_CONCURRENT: usize = 4; // Límite N de threads activos

// Simulación de la petición con latencia variable
fn simulate_request() -> Duration {
    let start = Instant::now();
    // Simular latencia variable entre 200ms y 1200ms
    let sleep_ms = thread_rng().gen_range(200..1200);
    thread::sleep(Duration::from_millis(sleep_ms));
    start.elapsed()
}

fn main() {
    // 1. Inicialización de Recursos Compartidos
    // El semáforo limita el acceso a MAX_CONCURRENT threads.
    let semaphore = Arc::new(Semaphore::new(MAX_CONCURRENT));
    // Mutex para proteger el almacenamiento de resultados (Estado Compartido).
    let all_times =
Arc::new(Mutex::new(Vec::<Duration>::with_capacity(N_SITES)));

    let mut handles = vec![];

    // 2. Fork (Creación de Tareas Concurrentes)
    for _id in 0..N_SITES {
        let sem_clone = Arc::clone(&semaphore);
        let times_clone = Arc::clone(&all_times);

        let handle = thread::spawn(move || {
            // A. INICIO DE ACCESO LIMITADO: Adquirir permiso
            (wait()).
            // Bloquea el thread si el semáforo está en 0.
            sem_clone.acquire();

```

```

        // ZONA LIMITADA (Máximo N threads ejecutan aquí
simultáneamente)
        let elapsed_time = simulate_request();

        // Acceso al estado compartido: guardar el resultado
times_clone.lock().unwrap().push(elapsed_time);

        // B. FIN DE ACCESO LIMITADO: Liberar permiso
(signal()).
        // Permite que otro thread bloqueado por el semáforo
continúe.
        sem_clone.release();
    });
    handles.push(handle);
}

// 3. Join (Esperar la Finalización)
for handle in handles {
    handle.join().unwrap();
}

// 4. Reducción (Cálculo del Promedio)
let final_times = all_times.lock().unwrap();
let total_duration: Duration = final_times.iter().sum();

// Calcular el promedio.
if let Some(avg_duration) = total_duration.checked_div(N_SITES
as u32) {
    println!("Tiempo promedio total de latencia: {:?}",
avg_duration);
}
}

```

[2c2023]

3. Armar un Semáforo usando Monitores.

```

// Pseudocódigo Rust
use std::sync::{Condvar, Mutex};

pub struct Semaphore {

```

```

    // 🔒 Mutex: Protege el contador V (recurso contable),
    asegurando exclusión mutua.
    recursos: Mutex<u32>,
    // 🚶 Condvar: Actúa como la cola de espera L, gestionando la
    espera pasiva.
    condvar: Condvar,
}

impl Semaphore {
    pub fn new(k: u32) -> Self {
        Self {
            recursos: Mutex::new(k),
            condvar: Condvar::new(),
        }
    }

    // metodo wait
    // Método wait() - Intentar ocupar un recurso (P)
    pub fn wait(&self) {
        let mut recursos_guard = self.recursos.lock().unwrap();
        // 1. Adquiere Mutex
        // 2. Lógica de Espera (Condición):
        // Siempre usar WHILE. Esto previene los "spurious
        wake-ups" (despertadas espurias).
        // El thread rechequea la condición (*recursos_guard == 0)
        al despertar [cite: 2756-2758].
        while *recursos_guard == 0 {
            // Llama a wait():
            // A. Libera el Mutex.
            // B. Bloquea el thread.
            // C. Al despertar, readquiere el Mutex.
            recursos_guard =
self.condvar.wait(recursos_guard).unwrap();
        }

        // 3. Consumo: Si sale del while, el recurso está
        disponible: V -= 1[cite: 2757].
        *recursos_guard -= 1;
    } // El Mutex es liberado aquí por RAII

    // Método signal() - Liberar o devolver el recurso (V)

```

```

pub fn signal(&self) {
    let mut recursos_guard = self.recursos.lock().unwrap();
    // 1. Adquiere Mutex

    // 2. Liberación: V += 1[cite: 2759].
    *recursos_guard += 1;

    // 3. Notificación: Despierta un thread que esté esperando
    en la cola (Condvar).
    self.condvar.notify_one();
} // El Mutex es liberado aquí por RAII
}

```

[2c2024]

3. Se quiere abrir un restaurante en el barrio de San Telmo. Se espera que los clientes lleguen y sean atendidos por alguno de los mozos de turno, cada uno de los cuales tomará los pedidos de cada mesa, los notificará a la cocina y luego seguirá tomando otros pedidos. Como la cocina es chica los cocineros pueden entrar a buscar los ingredientes al depósito de a uno a la vez, y buscar entre los distintos alimentos les puede llevar un tiempo variable. Cuando los cocineros hayan terminado de preparar un pedido deben notificar a los mozos para que lo lleven a la mesa. Además, los mozos deben estar disponibles para cobrarle a los clientes. Diseñe el sistema utilizando el modelo de actores, y para cada entidad defina cuáles son los estados internos y los mensajes que intercambian.

Mozos: pedidos_tomados: Map<MesaID, Pedido>, disponible: bool
TomarPedido(Pedido, ClienteAddr): Toma el pedido, lo registra y envía el mensaje **NotificarPedido** a la Cocina. Luego, sigue tomando otros pedidos. **PedidoListo(Comida)**: Recibe el plato de la Cocina y lo lleva al Cliente. **Cobrar(Monto)**: Procesa el pago del Cliente.

Depósito: ocupado: bool, cola_espera: Vec<Addr<Cocinero>>
SolicitarAcceso(CocineroAddr): **Serializa el acceso**. Si ocupado es false, lo marca y responde AccesoConcedido. Si está ocupado,

añade al Cocinero a la cola_espera. **Liberar()**: Si la cola no está vacía, saca al siguiente de la espera y le da acceso; si está vacía, pone ocupado = false

Cocineros: pedido_actual: Option<Pedido>, deposito: Addr<Deposito>
Preparar(Pedido): Si no está ocupado, toma el pedido. Envía **SolicitarAcceso()** al Depósito y espera la respuesta para buscar ingredientes. Al terminar de preparar, envía PedidoListo al Mozo correspondiente.

Cocina:cola_pedidos: Vec<Pedido>
NotificarPedido(Pedido, MozoAddr): Recibe el pedido del Mozo, lo añade a su cola interna y notifica al *pool* de Cocineros que hay trabajo.

Cliente: es el disparador y el que recibe el último mensaje

Cliente -> Mozo HacerPedido(Items)Inicia la transacción del servicio.
Mozo -> Cocina NotificarPedido(Pedido, MozoAddr)Transfiere el trabajo a la cocina, incluyendo su dirección para la respuesta.
Cocinero -> Depósito SolicitarAcceso()Petición para adquirir el recurso limitado (Exclusión Mutua).
Depósito -> Cocinero AccesoConcedido()Respuesta que actúa como el token de acceso para buscar los ingredientes.
Cocinero -> Mozo PedidoListo(Comida)Notificación para que el Mozo finalice el servicio al Cliente.
Cliente -> Mozo PedirCuenta(Monto)Solicita la transacción de cobro.

[2c2024-recu1]

Ejercicio 5

Nos encontramos implementando un sistema para una cafetería de especialidad. Los clientes llegarán al mostrador y serán atendidos por algún cajero para tomar su pedido y cobrarlo. Los baristas irán tomando los pedidos ingresados y los prepararán. Por el momento se disponen únicamente de dos máquinas de café, las cuales solo puede utilizar un barista por vez. Finalmente, al tener el pedido listo, el barista llamará al cliente por su nombre para entregárselo.

Diseñe el sistema utilizando el modelo de actores. Para cada uno de ellos especifique su estado interno en pseudocódigo de Rust. Detalle los mensajes que reciben, sus payloads y cómo los procesan también en pseudocódigo.

Tenemos como parte de este sistema una cola de pedidos. El recurso limitado en este caso son las dos maquinas de cafe dado que solo las puede utilizar un barista a la vez.

Vamos a definir el actor cliente, barista, cajero y máquina de café.

Cajero

- Estado interno:
cola_pedidos: Vec<Pedido>, baristas: Vec<Addr<Barista>>
Mensaje recibido: NuevoPedido(Pedido, ClienteAddr): toma el pedido del cliente y lo pone en la cola de pedidos y notifica a los baristas que hay un nuevo pedido.

Barista

- Estado interno:
pedido_actual: Option<Pedido>, maquinas: Map<ID, Addr<Maquina>
TomarTrabajo(): Adquiere un pedido de la cola_pedidos del Cajero. Intenta adquirir una máquina libre enviando **SolicitarUso** a una de ellas. **UsoAceptado(MaquinaID)**: Inicia la preparación. Al terminar, envía PedidoListo al Cliente.

Cliente

- Estado interno: nombre: String, pedido: Pedido
PedidoListo(Comida): Recibe el pedido final del Barista.

Máquina de café

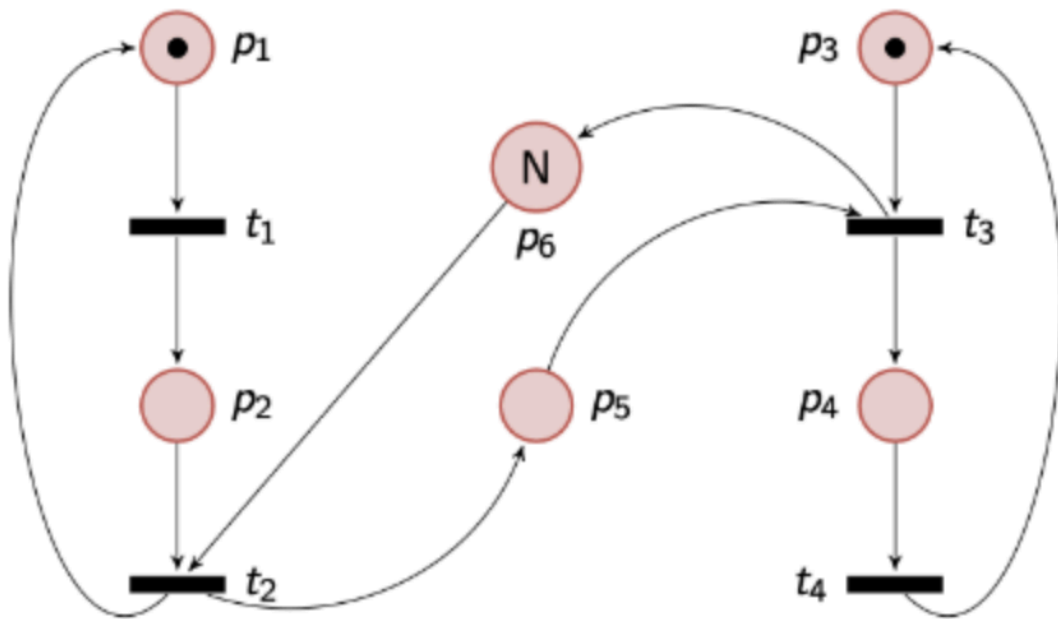
- Estado interno: ocupado_por: Option<Addr<Barista>>
SolicitarUso(BaristaAddr): **Exclusión Mutua**; si ocupado_por es None, marca la ocupación y responde UsoAceptado. Si está ocupada, responde Ocupada. **Liberar()**: Pone ocupado_por = None y notifica a los baristas en espera.

Redes de petri

-> Modelar la atención de los servidores
[2c2023]

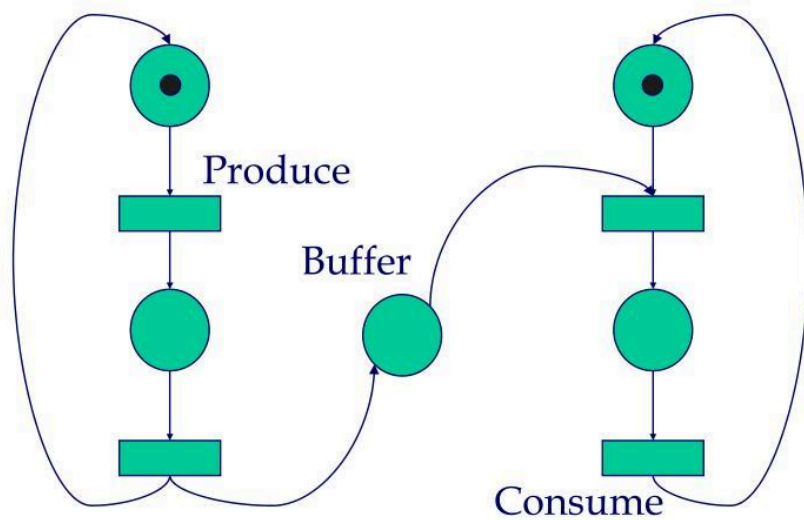
4. Dibujar la red de Petri para el problema de productor-consumidor con buffer acotado.

Productor, consumidor
opción buffer acortado



opción buffer no acotado

Producer-Consumer Problem



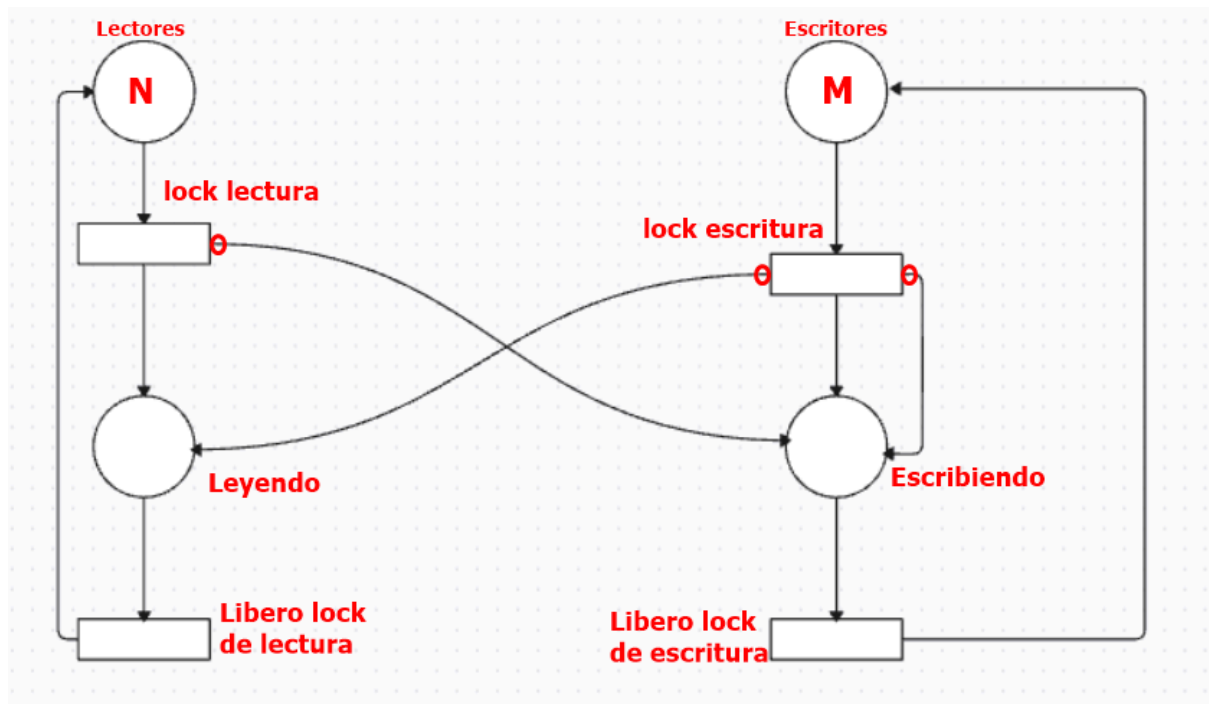
2. Modelar una Red de Petri para el problema del Lector-Escritor sin preferencia. Luego, modele una solución que contemple preferencia de escritura.

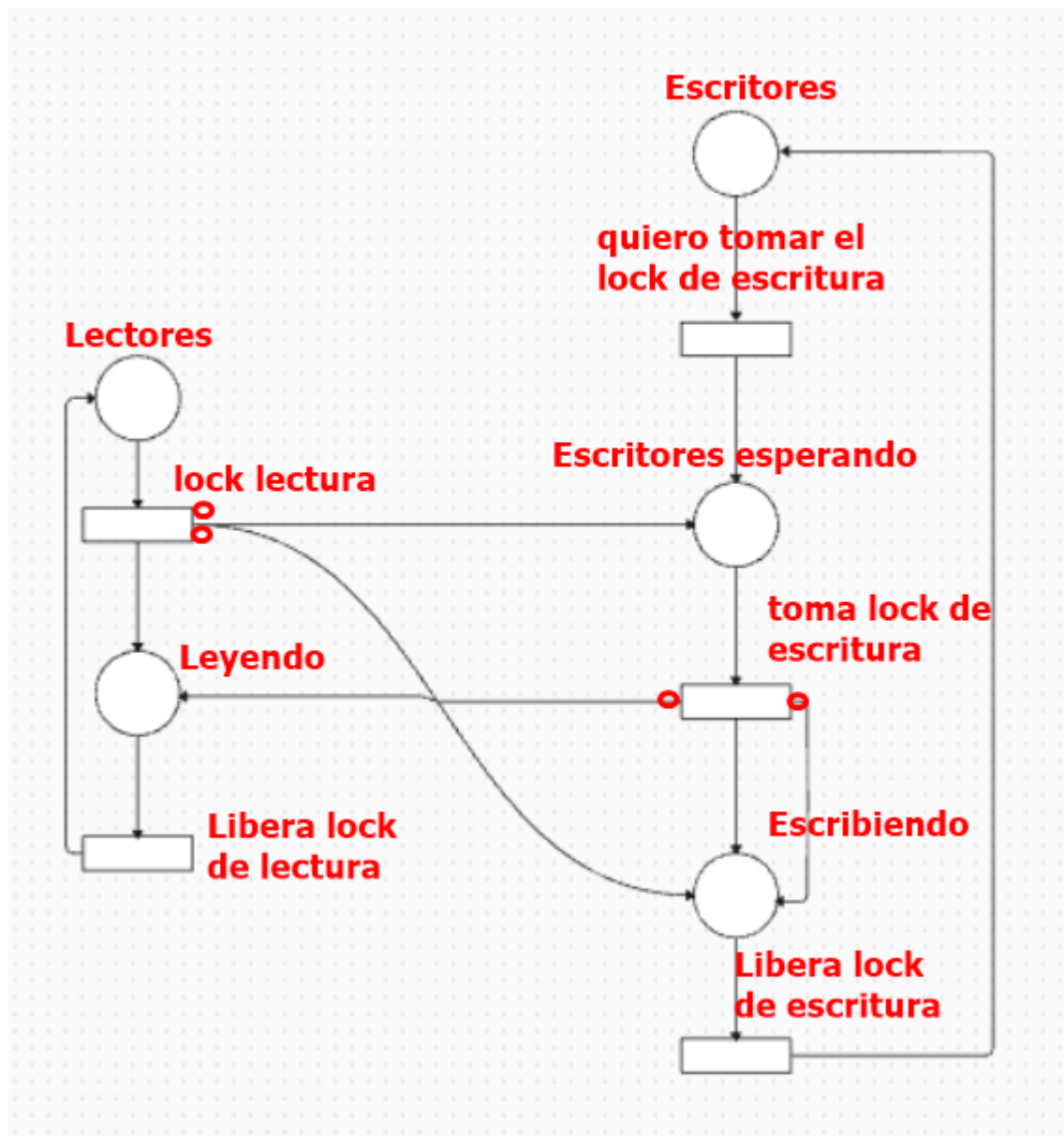
Lector, escritor

Tareas: lectura, escritura

¿Qué quiere decir sin preferencia?

Opción con preferencia





[2c2024-recu1]

Ejercicio 3

Modelar una Red de Petri para la solución propuesta en el punto anterior

En una revisión de código se encontró el siguiente fragmento

```
const N:usize = 10;
fn main() {

    let buffer = Arc::new(Mutex::new(Vec::<u32>::with_capacity(N)));

    let buffer_local = buffer.clone();
    let handle = thread::spawn(move || {
        loop {
            let mut buf = buffer_local.lock().unwrap();
            if buf.len() < N {
                buf.push(rand::thread_rng().gen());
            } else {
                drop(buf);
                thread::sleep(Duration::from_secs(1));
            }
        }
    });

    loop {
        let mut buf = buffer.lock().unwrap();
        if !buf.is_empty() {
            println!("{}", buf.pop().unwrap());
        } else {
            drop(buf);
            thread::sleep(Duration::from_secs(1));
        }
    }

    handle.join().unwrap();
}
```

- Describe qué problemas presenta este código y proponga una solución
- Describe por qué son necesarias las llamadas a drop en esta implementación.

Recordar:

Problemas más comunes

- Filósofos
- Banquero
- Peluquero
- Lector-escriptor
- Productor-consumidor

Modelos de concurrencia

- Actores
 - Función ppal:
 - Ventajas
 - Desventajas
- Programación asincrónica
 - Función ppal:
 - Ventajas
 - Desventajas

- Fork-join
 - Función ppal:
 - Ventajas
 - Desventajas
 - Estado mutable compartido
 - Función ppal:
 - Ventajas
 - Desventajas
 - Canales-mensajes
 - Función ppal:
 - Ventajas
 - Desventajas
-

● Condiciones para que NO se dé un Busy Wait (Espera Pasiva Correcta)

Un hilo **NO** está en *busy wait* si se suspende o cede el control del CPU de forma adecuada.

1. **Uso de Bloqueo Pasivo (Sincronización):** El código utiliza primitivas de sincronización que suspenden el *thread* hasta recibir una señal.
 - **Ejemplos:** `Mutex::lock()`, `Semaphore::acquire()`, `Condvar::wait()`, o `rx.recv()`.
 - **Razón:** El *thread* se pone en estado "bloqueado" por el sistema operativo, **consumiendo cero CPU** mientras espera la liberación del recurso .
2. **Cesión de CPU Explícita:** El hilo cede su turno después de realizar su trabajo productivo.
 - **Ejemplo:**
`thread::sleep(Duration::from_millis(DELAY_LARGO))`.
 - **Razón:** Aunque no es una espera de condición, el hilo renuncia a su turno por un tiempo significativo .
3. **Adquisición Mixta Exitosa (Problema del Filósofo):** El hilo adquiere los recursos necesarios sin fallar el `try_acquire()`.
 - **Razón:** El hilo pasa la mayor parte del tiempo bloqueado esperando pasivamente en el primer recurso (`acquire()`) o durmiendo, no girando en el bucle de reintento.

● Condiciones para que SÍ se dé un Busy Wait (Espera Activa)

Un hilo SÍ está en *busy wait* si se encuentra atrapado en un **bucle de verificación rápida** de una condición que no se resuelve de inmediato.

1. **El Patrón de Alerta (Bucle de Verificación):** El código contiene un `loop` o `while` infinito que encierra una verificación de estado:
 - **Ejemplos:** `loop { if recurso_listo { break } }` o `while !condicion_lista { ... }`.
 - **Razón:** El *thread* gasta ciclos de CPU **preguntando activamente** si la condición se cumple, en lugar de esperar una notificación.
2. **Reintento Rápido con Falla (Try/Catch):** El *thread* intenta adquirir un recurso no bloqueante (`try_acquire()`) o hacer una conexión externa (`TcpStream::connect()`), y si falla:
 - **Reacción 1 (Filósofo):** El hilo libera el recurso que tenía y el `loop` lo fuerza a **reintentar inmediatamente** .
 - **Reacción 2 (Consumidor/Red):** El hilo hace un `thread::sleep(DELAY_CORTO)` y **vuelve a chequear** .
 - **Razón:** En ambos casos, el *thread* está gastando CPU en la gestión del reintento, lo cual es ineficiente y es la definición de espera activa.
3. **Falta de `Condvar` en Estado Compartido:** Se utiliza solo un `Mutex` para proteger el estado compartido, pero no se usa una `Condvar` para notificar un cambio de condición (ej., el *buffer* no está vacío).
 - **Razón:** La ausencia de `Condvar` obliga a los hilos a usar la única alternativa: **chequear el estado en un bucle** (Busy Wait) .